

Perspective-n-Point

Perspective-*n*-Point^[1] (PnP) is the problem of estimating the pose of a camera given a set of *n* 3D points in the world and their corresponding 2D projections in an image. The camera pose consists of 6 degrees of freedom (DOF) which are made up of the rotation (roll, pitch, and yaw) and 3D translation of the camera with respect to an origin point in the world. This problem originates from camera calibration and has many applications in computer vision and other areas, including 3D tracking, robotics and augmented reality.^[2] A commonly used solution to the problem exists for *n* = 3 called P3P, and many solutions are available for the general case of *n* ≥ 3. A solution for *n* = 2 exists if feature orientations are available at the two points.^[3] Implementations of these solutions are available in open source software.

Problem specification

Definition

Given a set of *n* 3D points in a world reference frame and their corresponding 2D image projections as well as the calibrated intrinsic camera parameters, determine the 6 DOF pose of the camera in the form of its rotation and translation with respect to the world. This follows the perspective projection model for cameras:

$$s p_c = K [R | T] p_w.$$

where $p_w = [x \ y \ z \ 1]^T$ is the homogeneous world point, $p_c = [u \ v \ 1]^T$ is the corresponding homogeneous image point, *K* is the matrix of intrinsic camera parameters, (where *f_x* and *f_y* are the scaled focal lengths, *γ* is the skew parameter which is sometimes assumed to be 0, and (*u*₀, *v*₀) is the principal point), *s* is a scale factor for the image point, and *R* and *T* are the desired 3D rotation and 3D translation of the camera (extrinsic parameters) that are being calculated. This leads to the following equation for the model:

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & \gamma & u_0 \\ 0 & f_y & v_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_1 \\ r_{21} & r_{22} & r_{23} & t_2 \\ r_{31} & r_{32} & r_{33} & t_3 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}.$$

Assumptions and data characteristics

There are a few preliminary aspects of the problem that are common to all solutions of PnP. The assumption made in most solutions is that the camera is already calibrated. Thus, its intrinsic properties are already known, such as the focal length, principal image point, skew parameter, and

other parameters. Some methods, such as $UPnP$ ^[4] or the Direct Linear Transform (DLT) applied to the projection model, are exceptions to this assumption as they estimate these intrinsic parameters as well as the extrinsic parameters which make up the pose of the camera that the original PnP problem is trying to find.

For each solution to PnP , the chosen point correspondences cannot be colinear. In addition, PnP can have multiple solutions, and choosing a particular solution would require post-processing of the solution set. RANSAC is also commonly used with a PnP method to make the solution robust to outliers in the set of point correspondences. $P3P$ methods assume that the data is noise free, most PnP methods assume Gaussian noise on the inlier set.

Methods

This following section describes two common methods that can be used to solve the PnP problem that are also readily available in open source software and how RANSAC can be used to deal with outliers in the data set.

P3P

When $n = 3$, the PnP problem is in its minimal form of $P3P$ and can be solved with three point correspondences. However, with just three point correspondences, $P3P$ yields up to four real, geometrically feasible solutions. For low noise levels a fourth correspondence can be used to remove ambiguity. The setup for the problem is as follows.

Let P be the center of projection for the camera, A , B , and C be 3D world points with corresponding images points u , v , and w . Let $X = |PA|$, $Y = |PB|$, $Z = |PC|$, $\alpha = \angle BPC$, $\beta = \angle APC$, $\gamma = \angle APB$, $p = 2 \cos \alpha$, $q = 2 \cos \beta$, $r = 2 \cos \gamma$, $a' = |AB|$, $b' = |BC|$, $c' = |AC|$. This forms triangles PBC , PAC , and PAB from which we obtain a sufficient equation system for $P3P$:

$$\begin{cases} Y^2 + Z^2 - YZp - b'^2 &= 0 \\ Z^2 + X^2 - XZq - c'^2 &= 0 \\ X^2 + Y^2 - XYr - a'^2 &= 0 \end{cases}$$

Solving the $P3P$ system results in up to four geometrically feasible real solutions for R and T . The oldest published solution dates to 1841.^[5] A recent algorithm for solving the problem as well as a solution classification for it is given in the 2003 *IEEE Transactions on Pattern Analysis and Machine Intelligence* paper by Gao, et al.^[6] An open source implementation of Gao's $P3P$ solver can be found in OpenCV's *calib3d* module in the *solvePnP* function.^[7] Several faster and more accurate versions have been published since, including Lambda Twist $P3P$ ^[8] which achieved state of the art performance in 2018 with a 50 fold increase in speed and a 400 fold decrease in numerical failures. Lambdatwist is available as open source in OpenMVG and at <https://github.com/midji/pnp>.

EPnP

Efficient PnP (EPnP) is a method developed by Lepetit, et al. in their 2008 International Journal of Computer Vision paper^[9] that solves the general problem of PnP for $n \geq 4$. This method is based on the notion that each of the n points (which are called reference points) can be expressed as a weighted sum of four virtual control points. Thus, the coordinates of these control points become the unknowns of the problem. It is from these control points that the final pose of the camera is solved for.

As an overview of the process, first note that each of the n reference points in the world frame, p_i^w , and their corresponding image points, p_i^c , are weighted sums of the four controls points, c_j^w and c_j^c respectively, and the weights are normalized per reference point as shown below. All points are expressed in homogeneous form.

$$\begin{aligned} p_i^w &= \sum_{j=1}^4 \alpha_{ij} c_j^w \\ p_i^c &= \sum_{j=1}^4 \alpha_{ij} c_j^c \\ \sum_{j=1}^4 \alpha_{ij} &= 1 \end{aligned}$$

From this, the derivation of the image reference points becomes

$$s_i p_i^{img} = K \sum_{j=1}^4 \alpha_{ij} c_j^c.$$

Where p_i^{img} is the image reference points with pixel coordinate $[u_i \quad v_i \quad 1]^T$. The homogeneous image control point has the form $c_j^c = [x_j^c \quad y_j^c \quad z_j^c]^T$. Rearranging the image reference point equation yields the following two linear equations for each reference point:

$$\begin{aligned} \sum_{j=1}^4 \alpha_{ij} f_x x_j^c + \alpha_{ij} (u_0 - u_i) z_j^c &= 0 \\ \sum_{j=1}^4 \alpha_{ij} f_y y_j^c + \alpha_{ij} (v_0 - v_i) z_j^c &= 0. \end{aligned}$$

Using these two equations for each of the n reference points, the system $Mx = 0$ can be formed where $x = [c_1^{c^T} \quad c_2^{c^T} \quad c_3^{c^T} \quad c_4^{c^T}]^T$. The solution for the control points exists in the null space of M and is expressed as

$$x = \sum_{i=1}^N \beta_i v_i$$

where N is the number of null singular values in M and each v_i is the corresponding right singular vector of M . N can range from 0 to 4. After calculating the initial coefficients β_i , the Gauss-Newton algorithm is used to refine them. The R and T matrices that minimize the reprojection error of the world reference points, p_i^w , and their corresponding actual image points p_i^c , are then calculated.

This solution has $O(n)$ complexity and works in the general case of PnP for both planar and non-planar control points. Open source software implementations of this method can be found in OpenCV's Camera Calibration and 3D Reconstruction module in the *solvePnP* function^[7] as well as from the code published by Lepetit, et al. at their website, CVLAB at EPFL (<http://cvlab.epfl.ch/EPnP/index.php>).^[10]

This method is not robust against outliers and generally compares poorly to RANSAC P3P followed by nonlinear refinement .

SQPnP

SQPnP was described by Terzakis and Lourakis in an ECCV 2020 paper.^[11] It is a non-minimal, non-polynomial solver which casts PnP as a non-linear quadratic program. SQPnP identifies regions in the parameter space of 3D rotations (i.e., the 8-sphere) that contain unique minima with guarantees that at least one among them is the global minimum. Each regional minimum is computed with sequential quadratic programming that is initiated at nearest orthogonal approximation matrices.

SQPnP has similar or even higher accuracy compared to state of the art polynomial solvers, is globally optimal and computationally very efficient, being practically linear in the number of supplied points n . An optimized C++ implementation is available (<https://github.com/terzakig/sqpnp>) on GitHub, which has also been ported to OpenCV and included in the Camera Calibration and 3D Reconstruction module (*SolvePnP* function).^[12]

Using RANSAC

PnP is prone to errors if there are outliers in the set of point correspondences. Thus, RANSAC can be used in conjunction with existing solutions to make the final solution for the camera pose more robust to outliers. An open source implementation of PnP methods with RANSAC can be found in OpenCV's Camera Calibration and 3D Reconstruction module in the *solvePnP**Ransac* function.^[12]

See also

- Camera resectioning

- [Random sample consensus](#)
- [Visual odometry](#)
- [python OpenCV solvePnP \(https://docs.opencv.org/4.x/d5/d1f/calib3d_solvePnP.html\)](https://docs.opencv.org/4.x/d5/d1f/calib3d_solvePnP.html)

References

1. Fischler, M. A.; Bolles, R. C. (1981). "Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography" (<https://doi.org/10.1145%2F358669.358692>). *Communications of the ACM*. **24** (6): 381–395. doi:10.1145/358669.358692 (<https://doi.org/10.1145%2F358669.358692>). S2CID 972888 (<https://api.semanticscholar.org/CorpusID:972888>).
2. Apple, ARKIT team (2018). "Understanding ARKit Tracking and Detection" (<https://developer.apple.com/videos/play/wwdc2018/610>). WWDC.
3. Fabbri, Ricardo; Giblin, Peter; Kimia, Benjamin (2012). "Camera Pose Estimation Using First-Order Curve Differential Geometry". *Computer Vision – ECCV 2012* (<https://rfabbri.github.io/stuff/fabbri-giblin-kimia-eccv2012-final-ext.pdf>) (PDF). Lecture Notes in Computer Science. Vol. 7575. pp. 231–244. doi:10.1007/978-3-642-33765-9_17 (https://doi.org/10.1007%2F978-3-642-33765-9_17). ISBN 978-3-642-33764-2. S2CID 15402824 (<https://api.semanticscholar.org/CorpusID:15402824>).
4. Penate-Sanchez, A.; Andrade-Cetto, J.; Moreno-Noguer, F. (2013). "Exhaustive Linearization for Robust Camera Pose and Focal Length Estimation". *IEEE Transactions on Pattern Analysis and Machine Intelligence*. **35** (10): 2387–2400. doi:10.1109/TPAMI.2013.36 (<https://doi.org/10.1109%2FTPAMI.2013.36>). hdl:2117/22931 (<https://hdl.handle.net/2117%2F22931>). PMID 23969384 (<https://pubmed.ncbi.nlm.nih.gov/23969384>). S2CID 9614348 (<https://api.semanticscholar.org/CorpusID:9614348>).
5. Quan, Long; Lan, Zhong-Dan (1999). "Linear N-Point Camera Pose Determination" (<https://hal.inria.fr/docs/00/59/01/05/PDF/Quan-pami99.pdf>) (PDF). *IEEE Transactions on Pattern Analysis and Machine Intelligence*.
6. Gao, Xiao-Shan; Hou, Xiao-Rong; Tang, Jianliang; Cheng, Hang-Fei (2003). "Complete Solution Classification for the Perspective-Three-Point Problem". *IEEE Transactions on Pattern Analysis and Machine Intelligence*. **25** (8): 930–943. doi:10.1109/tpami.2003.1217599 (<https://doi.org/10.1109%2Ftpami.2003.1217599>). S2CID 15869446 (<https://api.semanticscholar.org/CorpusID:15869446>).
7. "Camera Calibration and 3D Reconstruction" (https://docs.opencv.org/master/d9/d0c/group_calib3d.html#ga549c2075fac14829ff4a58bc931c033d). *OpenCV*.
8. Persson, Mikael; Nordberg, Klas (2018). "Lambda Twist: An Accurate Fast Robust Perspective Three Point (P3P) Solver" (https://openaccess.thecvf.com/content_ECCV_2018/papers/Mikael_Persson_Lambda_Twist_An_ECCV_2018_paper.pdf) (PDF). *The European Conference on Computer Vision (ECCV)*.
9. Lepetit, V.; Moreno-Noguer, M.; Fua, P. (2009). "EPnP: An Accurate O(n) Solution to the PnP Problem". *International Journal of Computer Vision*. **81** (2): 155–166. doi:10.1007/s11263-008-0152-6 (<https://doi.org/10.1007%2Fs11263-008-0152-6>). hdl:2117/10327 (<https://hdl.handle.net/2117%2F10327>). S2CID 207252029 (<https://api.semanticscholar.org/CorpusID:207252029>).
10. "EPnP: Efficient Perspective-n-Point Camera Pose Estimation" (<http://cvlab.epfl.ch/EPnP/index.php>). *EPFL-CVLAB*.
11. Terzakis, George; Lourakis, Manolis (2020). "A Consistently Fast and Globally Optimal Solution to the Perspective-n-Point Problem". *Computer Vision – ECCV 2020* (https://www.ecva.net/papers/eccv_2020/papers_ECCV/html/1969_ECCV_2020_paper.php). Lecture Notes in Computer Science. Vol. 12346. pp. 478–494. doi:10.1007/978-3-030-58452-8_28 (https://doi.org/10.1007%2F978-3-030-58452-8_28). ISBN 978-3-030-58451-1. S2CID 226239551 (<https://api.semanticscholar.org/CorpusID:226239551>).

12. "Camera Calibration and 3D Reconstruction" (https://docs.opencv.org/master/d9/d0c/group__calib3d.html#ga50620f0e26e02caa2e9adc07b5fbf24e). *OpenCV*.

External links

- OpenCV Camera Calibration and 3D Reconstruction Module (https://docs.opencv.org/master/d9/d0c/group__calib3d.html#gsc.tab=0)
 - EPFL's CVLAB: EPnP (<http://cvlab.epfl.ch/EPnP/index.php>)
-

Retrieved from "<https://en.wikipedia.org/w/index.php?title=Perspective-n-Point&oldid=1361824110>"